

**LAPORAN PRAKTIKUM
PEMROGRAMAN PERANGKAT BERGERAK**

**MODUL 14
DATA STORAGE (API)**



**Disusun Oleh :
Dzikri Naufal Wisnu Pravida/2211104063
SE06-02**

**Asisten Praktikum :
Muhammad Faza Zulian
Aisyah Hasna Aulia**

**Dosen Pengampu :
Yudha Islami Sulistya**

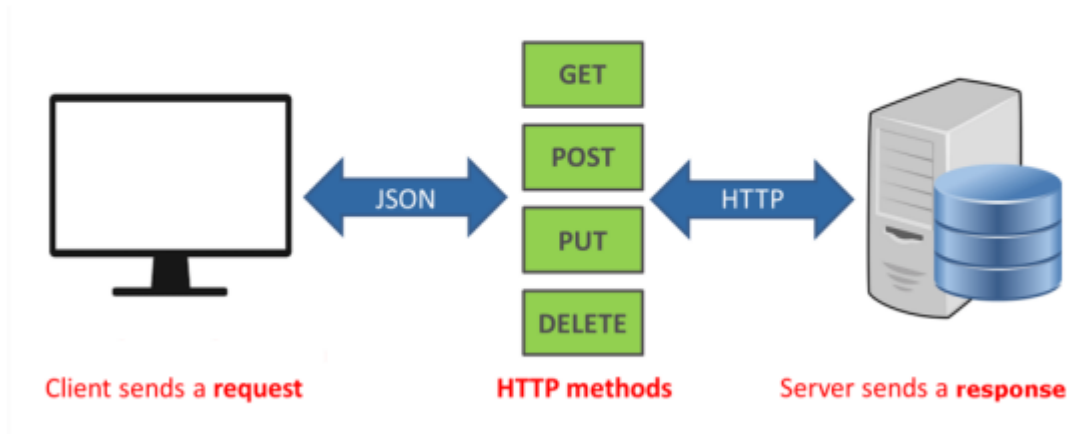
**PROGRAM STUDI S1 SOFTWARE ENGINEERING
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024**

PRAKTIKUM

A. GUIDED

1. Teori

Data Storage dengan Rest API



Gambar 1. Cara Kerja Restful API

1.1. Rest API

REST API (Representational State Transfer Application Programming Interface) adalah antarmuka yang memungkinkan aplikasi klien (misalnya, aplikasi web atau mobile) untuk berkomunikasi dengan server melalui protokol HTTP. REST API memungkinkan pengelolaan data seperti membaca, menambah, memperbarui, dan menghapus informasi dari database secara efisien dan tanpa harus mengakses database secara langsung.

Keunggulan REST API:

- Interoperabilitas: Memungkinkan berbagai aplikasi berbasis web dan mobile untuk mengakses data yang sama.
- Efisiensi: Data biasanya dikirim dalam format ringan seperti JSON atau XML, sehingga proses komunikasi lebih cepat.
- Keamanan: Mendukung token autentikasi untuk membatasi akses data.

REST API menggunakan metode HTTP seperti GET, POST, PUT, dan DELETE untuk melakukan operasi pada data.

1.2. HTTP

HTTP (Hypertext Transfer Protocol) adalah protokol utama yang digunakan untuk komunikasi antara klien (seperti browser atau aplikasi) dan server. HTTP bertanggung jawab untuk mengirim dan menerima data melalui internet.

Komponen HTTP:

1. Request:

- URL: Alamat sumber daya (resource).
 - Method: Operasi yang ingin dilakukan, seperti GET, POST, dll.
 - Headers: Informasi tambahan, seperti format data atau token autentikasi.
 - Body: Data yang dikirim (digunakan dalam POST/PUT).
2. Response:
- Status Code: Menunjukkan hasil operasi (misalnya, 200 OK, 404 Not Found).
 - Headers: Informasi tambahan dari server.
 - Body: Data yang dikembalikan server (seringkali dalam format JSON).
- Metode HTTP:
- GET: Mengambil data dari server.
 - POST: Mengirim data baru ke server.
 - PUT/PATCH: Memperbarui data yang ada di server.
 - DELETE: Menghapus data dari server.

2. Hasil Praktikum

Source Code

Main

```

1  import 'package:flutter/material.dart';
2  import 'package:prak_14/screen/home_screen.dart';
3
4  void main() {
5    runApp(const MyApp());
6  }
7
8  class MyApp extends StatelessWidget {
9    const MyApp({super.key});
10
11    // This widget is the root of your application.
12    @override
13    Widget build(BuildContext context) {
14      return MaterialApp(
15        title: 'Flutter Demo',
16        theme: ThemeData(
17          colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
18          useMaterial3: true,
19        ),
20        home: HomePageScreen(),
21      );
22    }
23  }
24

```

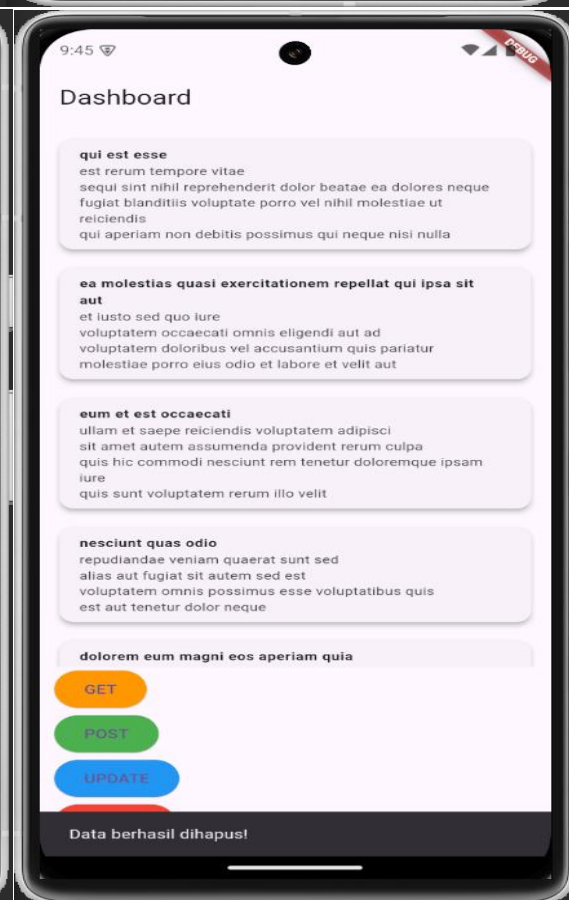
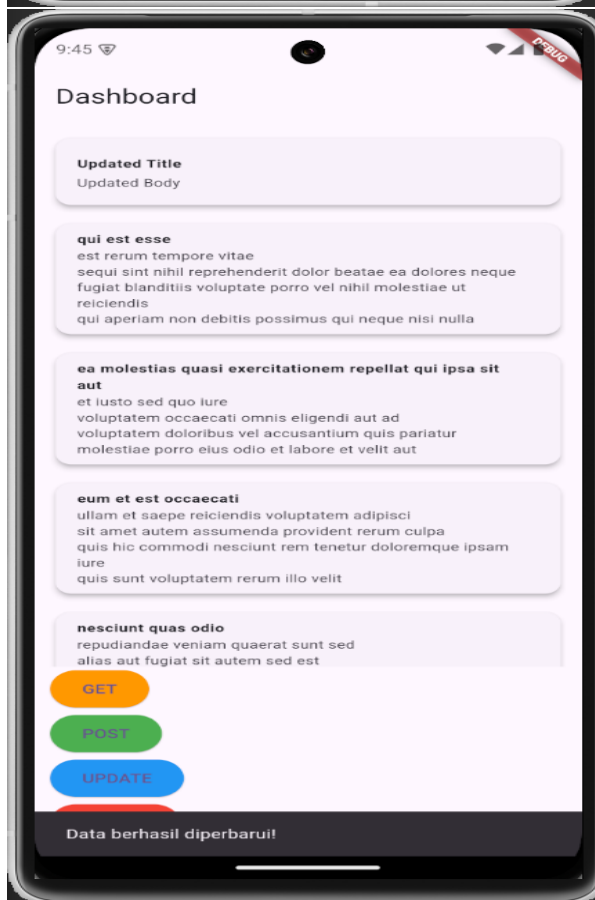
Homescreen

```
1 import 'package:flutter/material.dart';
2 import 'package:prak_14/service/api_service.dart';
3
4 class HomePageScreen extends StatefulWidget {
5   const HomePageScreen({super.key});
6
7   @override
8   State<HomePageScreen> createState() => _HomePageScreenState();
9 }
10
11 class _HomePageScreenState extends State<HomePageScreen> {
12   List<dynamic> _posts = []; // Menyimpan list posts
13   bool _isLoading = false; // Untuk indikator loading
14   final ApiService _apiService = ApiService(); // Instance ApiService
15   // Fungsi untuk menampilkan Snackbar
16   void _showSnackBar(String message) {
17     ScaffoldMessenger.of(context)
18       .showSnackBar(SnackBar(content: Text(message)));
19   }
20
21   // Fungsi untuk memanggil API dan menangani operasi
22   Future<void> _handleApiOperation(
23     Future<void> operation, String successMessage) async {
24     setState(() {
25       _isLoading = true;
26     });
27
28     try {
29       await operation; // Menjalankan operasi API
30       setState(() {
31         _posts = _apiService.posts;
32       });
33       _showSnackBar(successMessage);
34     } catch (e) {
35       _showSnackBar('Error: $e');
36     } finally {
37       setState(() {
38         _isLoading = false;
39       });
40     }
41   }
42
43   @override
44   Widget build(BuildContext context) {
45     return Scaffold(
46       appBar: AppBar(
47         title: const Text("Dashboard"),
48       ),
49       body: Padding(
50         padding: const EdgeInsets.all(12),
51         child: Column(
52           crossAxisAlignment: CrossAxisAlignment.start,
53           children: [
54             _isLoading
55               ? const Center(child: CircularProgressIndicator())
56               : _posts.isEmpty
57                 ? const Text(
58                     "Tekan tombol GET untuk mengambil data",
59                     style: TextStyle(fontSize: 12),
60                   )
61                 : Expanded(
62                     child: ListView.builder(
63                       itemCount: _posts.length,
64                       itemBuilder: (context, index) {
65                         return Padding(
66                           padding: const EdgeInsets.only(bottom: 12.0),
67                           child: Card(
68                             elevation: 4,
69                             child: ListTile(
70                               title: Text(
71                                 _posts[index]['title'],
72                                 style: const TextStyle(
73                                   fontWeight: FontWeight.bold,
74                                   fontSize: 12),
75                               ),
76                               subtitle: Text(
77                                 _posts[index]['body'],
78                                 style: const TextStyle(fontSize: 12),
79                               ),
80                             ),
81                         ),
82                       ),
83                     ),
84                   ),
85                 ElevatedButton(
86                   onPressed: () => _handleApiOperation(
87                     _apiService.fetchPosts(), 'Data berhasil diambil!'),
88                     style: ElevatedButton.styleFrom(backgroundColor: Colors.orange),
89                     child: const Text('GET'),
90                   ),
91                 ElevatedButton(
92                   onPressed: () => _handleApiOperation(
93                     _apiService.createPost(), 'Data berhasil ditambahkan!'),
94                     style: ElevatedButton.styleFrom(backgroundColor: Colors.green),
95                     child: const Text('POST'),
96                   ),
97                 ElevatedButton(
98                   onPressed: () => _handleApiOperation(
99                     _apiService.updatePost(), 'Data berhasil diperbarui!'),
100                     style: ElevatedButton.styleFrom(backgroundColor: Colors.blue),
101                     child: const Text('UPDATE'),
102                   ),
103                 ElevatedButton(
104                   onPressed: () => _handleApiOperation(
105                     _apiService.deletePost(), 'Data berhasil dihapus!'),
106                     style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
107                     child: const Text('DELETE'),
108                   ),
109               ),
110             ),
111           ],
112         ),
113       ),
114     );
115   }
116 }
```

Api service

```
1 import 'dart:convert';
2 import 'package:http/http.dart' as http;
3
4 class ApiService {
5   final String baseUrl = "https://jsonplaceholder.typicode.com";
6   List<dynamic> posts = []; // Menyimpan data post yang diterima
7
8   // Fungsi untuk GET data
9   Future<void> fetchPosts() async {
10    final response = await http.get(Uri.parse('$baseUrl/posts'));
11    if (response.statusCode == 200) {
12      posts = json.decode(response.body);
13    } else {
14      throw Exception('Failed to load posts');
15    }
16  }
17
18   // Fungsi untuk POST data
19   Future<void> createPost() async {
20    final response = await http.post(
21      Uri.parse('$baseUrl/posts'),
22      headers: {'Content-Type': 'application/json'},
23      body: json.encode({
24        'title': 'Flutter Post',
25        'body': 'Ini contoh POST.',
26        'userId': 1,
27      })),
28    );
29    if (response.statusCode == 201) {
30      posts.add({
31        'title': 'Flutter Post',
32        'body': 'Ini contoh POST.',
33        'id': posts.length + 1,
34      });
35    } else {
36      throw Exception('Failed to create post');
37    }
38  }
39
40   // Fungsi untuk UPDATE data
41   Future<void> updatePost() async {
42    final response = await http.put(
43      Uri.parse('$baseUrl/posts/1'),
44      body: json.encode({
45        'title': 'Updated Title',
46        'body': 'Updated Body',
47        'userId': 1,
48      })),
49    );
50    if (response.statusCode == 200) {
51      final updatedPost = posts.firstWhere((post) => post['id'] == 1);
52      updatedPost['title'] = 'Updated Title';
53      updatedPost['body'] = 'Updated Body';
54    } else {
55      throw Exception('Failed to update post');
56    }
57  }
58
59   // Fungsi untuk DELETE data
60   Future<void> deletePost() async {
61    final response = await http.delete(
62      Uri.parse('$baseUrl/posts/1'),
63    );
64    if (response.statusCode == 200) {
65      posts.removeWhere((post) => post['id'] == 1);
66    } else {
67      throw Exception('Failed to delete post');
68    }
69  }
70 }
71
```

Output :



B. UNGUIDED

Modifikasi tampilan Guided dari praktikum di atas:

a. Gunakan State Management dengan GetX:

- Atur data menggunakan state management GetX agar lebih mudah dikelola.
- Implementasi GetX meliputi pembuatan controller untuk mengelola data dan penggunaan widget Obx untuk menampilkan data secara otomatis setiap kali ada perubahan.

b. Tambahkan Snackbar untuk Memberikan Respon Berhasil:

- Tampilkan snackbar setelah setiap operasi berhasil, seperti menambah atau memperbarui data.
- Gunakan Get.snackbar agar pesan sukses muncul di layar dan mudah dipahami oleh pengguna.

Sourcecode

Main

```
1  import 'package:flutter/material.dart';
2  import 'package:get/get.dart';
3  import 'package:prak_14/screen/home_screen.dart';
4
5  void main() {
6    runApp(const MyApp());
7  }
8
9  class MyApp extends StatelessWidget {
10    const MyApp({super.key});
11
12    @override
13    Widget build(BuildContext context) {
14      return GetMaterialApp(
15        // Changed to GetMaterialApp
16        title: 'Flutter Demo',
17        theme: ThemeData(
18          colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
19          useMaterial3: true,
20        ),
21        home: HomePageScreen(),
22      );
23    }
24  }
```

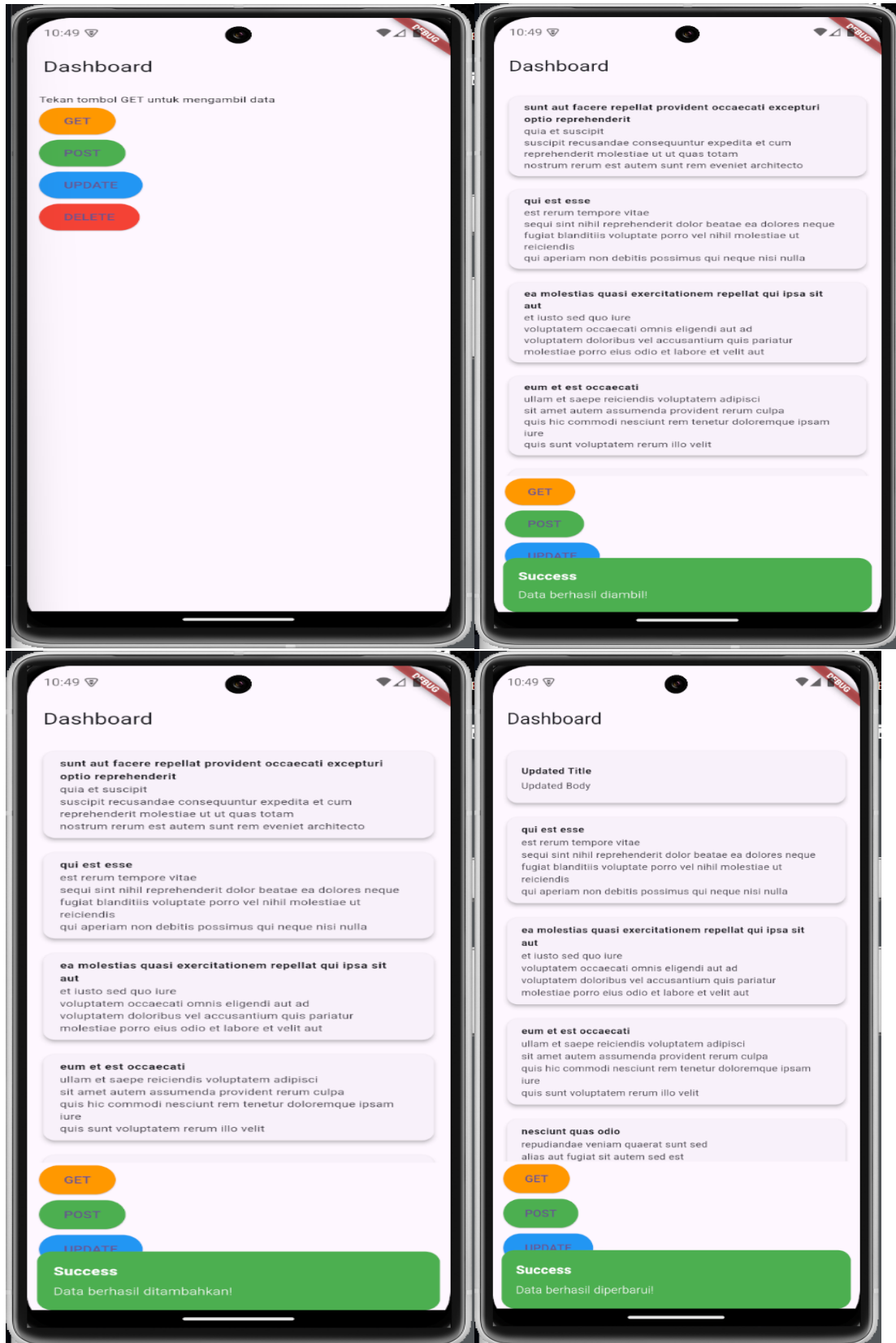
Homescreen

```
1 import 'package:flutter/material.dart';
2 import 'package:get/get.dart';
3 import 'package:prak_14/service/api_service.dart';
4
5 class HomePageScreen extends StatelessWidget {
6   HomePageScreen({super.key});
7
8   final ApiService apiService = Get.put(ApiService()); // Initialize controller
9
10  @override
11  Widget build(BuildContext context) {
12    return Scaffold(
13      appBar: AppBar(
14        title: const Text("Dashboard"),
15      ),
16      body: Padding(
17        padding: const EdgeInsets.all(12),
18        child: Column(
19          mainAxisAlignment: MainAxisAlignment.start,
20          children: [
21            Obx(() {
22              // Observe changes in loading and posts
23              if (apiService.isLoading.value) {
24                return const Center(child: CircularProgressIndicator());
25              }
26              if (apiService.posts.isEmpty) {
27                return const Text(
28                  "Tekan tombol GET untuk mengambil data",
29                  style: TextStyle(fontSize: 12),
30                );
31              }
32              return Expanded(
33                child: ListView.builder(
34                  itemCount: apiService.posts.length,
35                  itemBuilder: (context, index) {
36                    return Padding(
37                      padding: const EdgeInsets.only(bottom: 12.0),
38                      child: Card(
39                        elevation: 4,
40                        child: ListTile(
41                          title: Text(
42                            apiService.posts[index]['title'],
43                            style: const TextStyle(
44                              fontWeight: FontWeight.bold,
45                              fontSize: 12,
46                            ),
47                        ),
48                          subtitle: Text(
49                            apiService.posts[index]['body'],
50                            style: const TextStyle(fontSize: 12),
51                          ),
52                        ),
53                      ),
54                    );
55                  },
56                ),
57              );
58            ),
59            ElevatedButton(
60              onPressed: () => apiService.fetchPosts(),
61              style: ElevatedButton.styleFrom(backgroundColor: Colors.orange),
62              child: const Text('GET'),
63            ),
64            ElevatedButton(
65              onPressed: () => apiService.createPost(),
66              style: ElevatedButton.styleFrom(backgroundColor: Colors.green),
67              child: const Text('POST'),
68            ),
69            ElevatedButton(
70              onPressed: () => apiService.updatePost(),
71              style: ElevatedButton.styleFrom(backgroundColor: Colors.blue),
72              child: const Text('UPDATE'),
73            ),
74            ElevatedButton(
75              onPressed: () => apiService.deletePost(),
76              style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
77              child: const Text('DELETE'),
78            ),
79          ],
80        ),
81      );
82    }
83  }
84 }
85
```

Api service

```
1 import 'dart:convert';
2 import 'package:flutter/material.dart';
3 import 'package:http/http.dart' as http;
4 import 'package:get/get.dart';
5
6 class ApiService extends GetxController {
7   final String baseUrl = "https://jsonplaceholder.typicode.com";
8   RxList posts = [].obs; // Changed to observable list
9   RxBool isLoading = false.obs; // Added loading state
10
11   // Fungsi untuk GET data
12   Future<void> fetchPosts() async {
13     try {
14       isLoading.value = true;
15       final response = await http.get(Uri.parse('$baseUrl/posts'));
16       if (response.statusCode == 200) {
17         posts.value = json.decode(response.body);
18         Get.snackbar(
19           'Success',
20           'Data berhasil diambil!',
21           snackPosition: SnackPosition.BOTTOM,
22           backgroundColor: Colors.green,
23           colorText: Colors.white,
24         );
25       } else {
26         throw Exception('Failed to load posts');
27       }
28     } catch (e) {
29       Get.snackbar(
30         'Error',
31         'Failed to load posts: $e',
32         snackPosition: SnackPosition.BOTTOM,
33         backgroundColor: Colors.red,
34         colorText: Colors.white,
35       );
36     } finally {
37       isLoading.value = false;
38     }
39   }
40
41   // Fungsi untuk POST data
42   Future<void> createPost() async {
43     try {
44       isLoading.value = true;
45       final response = await http.post(
46         Uri.parse('$baseUrl/posts'),
47         headers: {'Content-Type': 'application/json'},
48         body: json.encode({
49           'title': 'Flutter Post',
50           'body': 'Ini contoh POST.',
51           'userId': 1,
52         })),
53       );
54       if (response.statusCode == 201) {
55         posts.add({
56           'title': 'Flutter Post',
57           'body': 'Ini contoh POST.',
58           'id': posts.length + 1,
59         });
60         Get.snackbar(
61           'Success',
62           'Data berhasil ditambahkan!',
63           snackPosition: SnackPosition.BOTTOM,
64           backgroundColor: Colors.green,
65           colorText: Colors.white,
66         );
67       } else {
68         throw Exception('Failed to create post');
69       }
70     } catch (e) {
71       Get.snackbar(
72         'Error',
73         'Failed to create post: $e',
74         snackPosition: SnackPosition.BOTTOM,
75         backgroundColor: Colors.red,
76         colorText: Colors.white,
77       );
78     } finally {
79       isLoading.value = false;
80     }
81   }
82
83   // Fungsi untuk UPDATE data
84   Future<void> updatePost() async {
85     try {
86       isLoading.value = true;
87       final response = await http.put(
88         Uri.parse('$baseUrl/posts/1'),
89         body: json.encode({
90           'title': 'Updated Title',
91           'body': 'Updated Body',
92           'userId': 1,
93         })),
94       );
95       if (response.statusCode == 200) {
96         final index = posts.indexWhere((post) => post['id'] == 1);
97         if (index != -1) {
98           posts[index] = {
99             ...posts[index],
100             'title': 'Updated Title',
101             'body': 'Updated Body',
102           };
103         }
104         Get.snackbar(
105           'Success',
106           'Data berhasil diperbarui!',
107           snackPosition: SnackPosition.BOTTOM,
108           backgroundColor: Colors.green,
109           colorText: Colors.white,
110         );
111       } else {
112         throw Exception('Failed to update post');
113       }
114     } catch (e) {
115       Get.snackbar(
116         'Error',
117         'Failed to update post: $e',
118         snackPosition: SnackPosition.BOTTOM,
119         backgroundColor: Colors.red,
120         colorText: Colors.white,
121       );
122     } finally {
123       isLoading.value = false;
124     }
125   }
126
127   // Fungsi untuk DELETE data
128   Future<void> deletePost() async {
129     try {
130       isLoading.value = true;
131       final response = await http.delete(
132         Uri.parse('$baseUrl/posts/1'),
133       );
134       if (response.statusCode == 200) {
135         posts.removeWhere((post) => post['id'] == 1);
136         Get.snackbar(
137           'Success',
138           'Data berhasil dihapus!',
139           snackPosition: SnackPosition.BOTTOM,
140           backgroundColor: Colors.green,
141           colorText: Colors.white,
142         );
143       } else {
144         throw Exception('Failed to delete post');
145       }
146     } catch (e) {
147       Get.snackbar(
148         'Error',
149         'Failed to delete post: $e',
150         snackPosition: SnackPosition.BOTTOM,
151         backgroundColor: Colors.red,
152         colorText: Colors.white,
153       );
154     } finally {
155       isLoading.value = false;
156     }
157   }
158 }
159
```

Screenshot Output



10:49

DEBUG

Dashboard

qui est esse

est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque
fugiat blanditiis voluptate porro vel nihil molestiae ut
reiciendis
qui aperiam non debitis possimus qui neque nisi nulla

ea molestias quasi exercitationem repellat qui ipsa sit aut

et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad
voluptatem doloribus vel accusantium quis pariatur
molestiae porro eius odio et labore et velit aut

eum et est occaecati

ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum culpa
quis hic commodi nesciunt rem tenetur doloremque ipsam
iure
quis sunt voluptatem rerum illo velit

nesciunt quas odio

repudiandae veniam quaerat sunt sed
alias aut fugiat sit autem sed est
voluptatem omnis possimus esse voluptatibus quis
est aut tenetur dolor neque

dolorem eum magni eos aperiam quia

GET

POST

UPDATE

Success

Data berhasil dihapus!

Deskripsi Program

Program ini adalah aplikasi Flutter berbasis GetX yang memanfaatkan REST API untuk mengelola data dari server menggunakan metode HTTP seperti GET, POST, PUT, dan DELETE. Komponen utama aplikasi terdiri dari ApiService sebagai pengelola logika API, dan HomePageScreen sebagai antarmuka pengguna (UI). Dengan mengintegrasikan GetX, aplikasi ini memiliki fitur reaktif, seperti menampilkan data yang diperoleh dari server secara otomatis menggunakan widget Obx tanpa perlu penyegaran manual. Pengguna dapat mengambil data, menambahkan entri baru, memperbarui, atau menghapus data melalui tombol-tombol yang tersedia di UI.

Setiap operasi API disertai umpan balik berupa SnackBar, yang memberikan notifikasi keberhasilan atau kegagalan. Data yang diambil dari server ditampilkan dalam format daftar (list), dengan elemen Card untuk setiap entri. Selain itu, aplikasi memanfaatkan pengelolaan status melalui GetX untuk menampilkan indikator pemrosesan, seperti CircularProgressIndicator, saat proses API berlangsung. Hal ini membuat aplikasi lebih dinamis dan responsif terhadap perubahan data secara real-time.